Forecasting: `engine/forecast.py`

Predicts the next 90 days of daily cost, and of fleet CPU % and memory %, with an additive Holt-Winters model written in plain NumPy (no statistics library), so every line can be explained.

1. What it does

Input: a series of daily values `y` (30 days). **Output (`holt_winters`):** the 90-day forecast, an uncertainty band for each day, and `sigma` (typical one-day error). **Accuracy check (`backtest`):** hold back the last 7 days, forecast them, report the mean absolute percentage error (MAPE).

It is used in `build.py` on three series: daily fleet cost (₹), daily average fleet CPU % and daily average fleet memory %.

2. Model

Additive Holt-Winters with a damped trend and a weekly season (period 7):

```
forecast(t) = level + φ × trend + season[t mod 7]
```

Starting values

- level = average of the first 7 days
- trend = (average of days 8–14 − average of days 1–7) ÷ 7
- season = each of the first 7 days minus the starting level

Update for each day `y_t` (with `s` the season value for that weekday):

```
level_new  = α × (y_t − s) + (1 − α) × (level + φ × trend)
trend_new  = β × (level_new − level_old) + (1 − β) × φ × trend
season_new = γ × (y_t − level_new) + (1 − γ) × s
```

Parameters (defaults in the function): α = 0.5 (level), β = 0.05 (trend), γ = 0.2 (season), φ = 0.8 (damping), season = 7, horizon = 90.

Damped trend: each future day adds `φ^h × trend`, so the slope fades out instead of growing forever. The forecast is `level + (φ + φ² + … + φ^h) × trend + season`.

3. Uncertainty band

- `sigma` = standard deviation of the one-step errors (actual − fitted) after the first week.
- band at horizon `h` days = `sigma × (1 + 0.15 × √h)`.
- Shown as `lo = forecast − band` and `hi = forecast + band`. For CPU and memory the values are clipped to 0–100%.

4. Backtest

If the history has at least 21 days (14 to fit + 7 to test): fit on all but the last 7 days, forecast the 7, and compute `MAPE = average( |forecast − actual| ÷ actual ) × 100`. Fewer than

21 days → no backtest.

5. When the forecast is shown

Only when the data has at least 14 days (`FORECAST_MIN_DAYS`). Below that the app shows a message explaining why (two weeks are needed to learn the weekly pattern).

6. Sample output (my run, seed 42)

- Cost forecast: backtest MAPE 2.4%, sigma ≈ ₹1,287 per day.
- Fleet CPU forecast MAPE 2.5%, memory 0.7%.

7. Assumptions

- Costs and utilisation follow a level, a gentle trend and a weekly rhythm.
- Past behaviour continues, damped.
- Daily fleet averages are a good summary of the whole estate.

8. Limitations

1. **Only 30 days of history.** That is about four weekly cycles, so the weekly pattern is barely learned. In the simulated data weekly seasonality is also weak.
2. **Structural breaks.** The 8 ML VMs launched on day 20 create a jump in cost. The model adapts its level, but it cannot know about planned launches, and it does not include the savings from accepted recommendations (the what-if simulator handles that separately).
3. **Heuristic band.** The `0.15 × Vh` widening is a simple rule of thumb, not a statistical prediction interval. Do not describe it as a 95% interval.
4. **Low MAPE is flattering.** A 7-day test on smooth simulated data is an easy test. Real data with promotions, incidents and monthly cycles would score worse.
5. **Parameters are fixed.** α, β, γ and φ are not tuned per series.
6. **Documentation mismatch to fix.** `ASSUMPTIONS.md` lists α 0.4, β 0.05, γ 0.3, but the code uses α 0.5, β 0.05, γ 0.2 and a damped trend (φ 0.8). Make the two agree before you submit.